Averon v4.0.0 — Full System Architecture

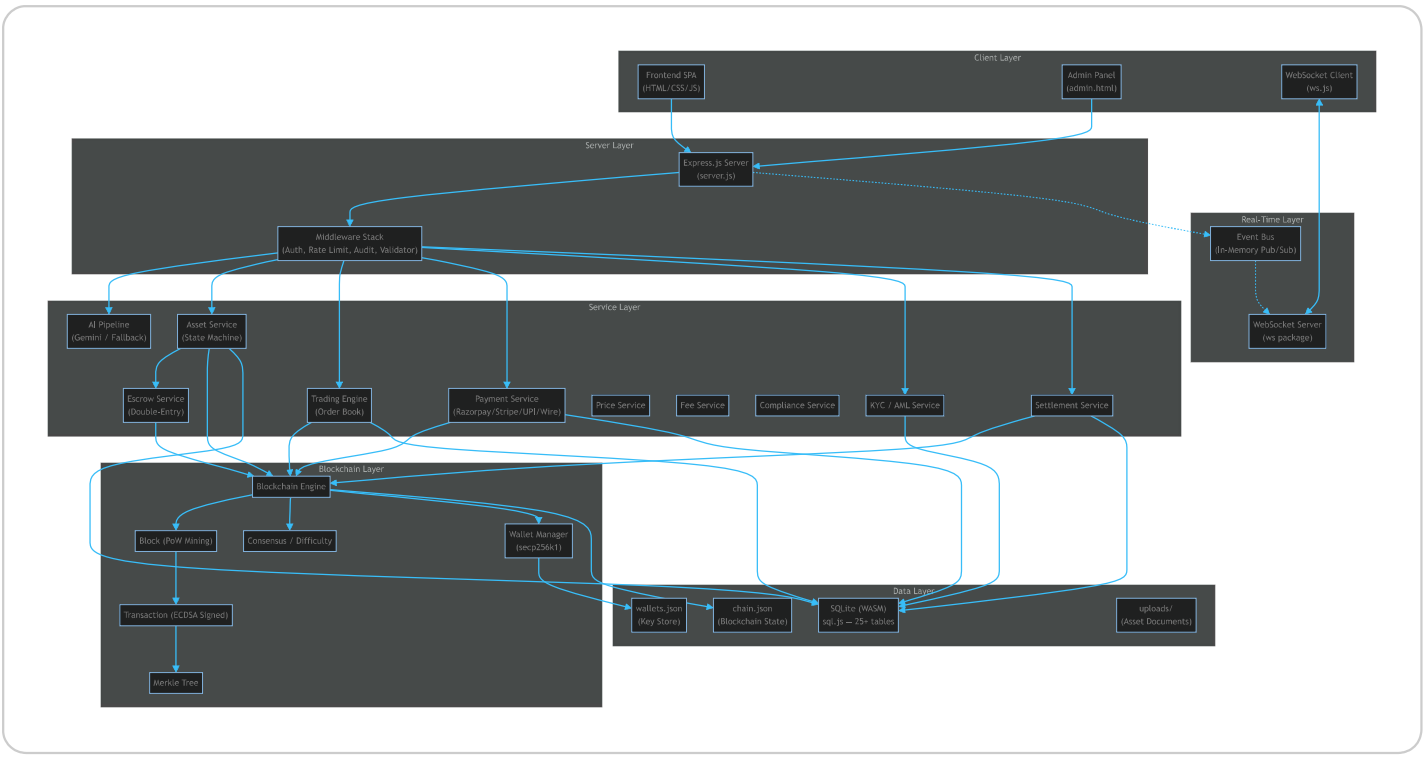
> *Enterprise Blockchain Asset Tokenization Platform with AI Analysis*

---

1. High-Level Overview

Averon is a full-stack **real-world asset (RWA) tokenization platform** that allows users to:

1. **Register** and receive a blockchain wallet (ECDSA secp256k1)
2. **Purchase Averon Coins (AC)** using fiat (INR) via Razorpay, Stripe, UPI, or Wire
3. **Create real-world assets** (land, vehicles, stocks, etc.) and upload supporting documents
4. **AI-analyze** those assets using Google Gemini (or a fallback engine) for verification, valuation, and risk scoring
5. **Tokenize** verified assets into fractional tokens on the blockchain
6. **Buy/sell asset tokens** using AC, with an escrow-backed funding mechanism
7. **Trade AC** on an integrated order-book exchange with circuit breakers
8. **Withdraw** — convert AC back to fiat with full KYC/AML compliance
9. **Admin panel** — full platform oversight, user management, compliance review



2. Technology Stack

Layer	Technology	Details
Runtime	Node.js ≥18	ES2022+, native `fetch` , `crypto`
Framework	Express.js 4.x	REST API server
Database	sql.js (SQLite WASM)	Zero native dependencies, in-process
Blockchain	Custom PoW chain	SHA-256 mining, Merkle trees, ECDSA
Cryptography	Node `crypto`	secp256k1 keys, PBKDF2, HMAC-SHA256
AI	Google Gemini 2.0 Flash	Multi-modal asset analysis
Payments	Razorpay + Stripe SDK	INR/USD/EUR payment processing

Layer	Technology	Details
WebSocket	`ws` package	Real-time price/trade/block updates
File Upload	Multer	Document storage for asset verification
Auth	Custom JWT (zero-dep)	HMAC-SHA256 signed, PBKDF2 passwords
Process Mgmt	PM2 / Node Cluster	Multi-worker production mode
Container	Docker + Compose	1GB memory limit, health checks
Frontend	Vanilla HTML/CSS/JS	SPA with WebSocket integration

3. Project Structure

```
d:\Averon\
├── server.js                    # Entry point — Express app, routes, init
├── package.json                 # v4.0.0, 7 dependencies
├── Dockerfile / docker-compose  # Production containerization
├── ecosystem.config.js          # PM2 cluster config
├── .env / .env.example          # Environment configuration
│
├── backend/
│   ├── config/
│   │   ├── constants.js         # ALL platform constants (264 lines)
│   │   ├── database.js          # SQLite WASM init, 25+ table schema (701 lines)
│   │   └── security.js          # Helmet/CORS security headers
│   │
│   ├── middleware/
│   │   ├── auth.js              # JWT + PBKDF2 + session management
│   │   ├── adminAuth.js         # Admin role enforcement
```

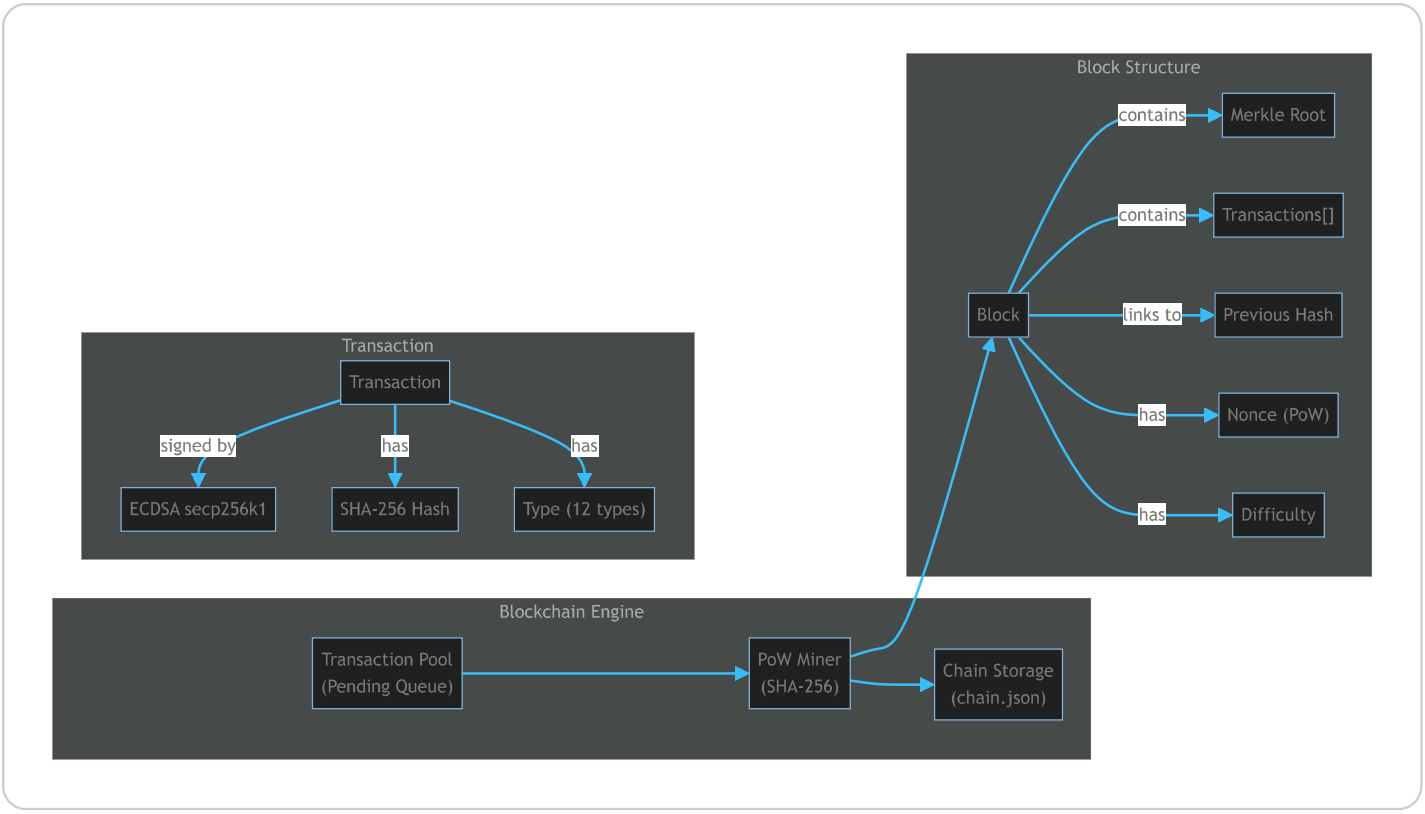
```
│   │   ├── audit.js            # Tamper-proof hash-chained audit log
│   │   ├── rateLimiter.js      # Token-bucket rate limiting (4 tiers)
│   │   └── validator.js        # Input validation & sanitization
│   │
│   ├── blockchain/
│   │   ├── chain.js            # Blockchain core (mining, balance, persistence)
│   │   ├── block.js            # Block structure with Merkle root
│   │   ├── transaction.js      # ECDSA-signed transactions
│   │   ├── merkle.js           # Merkle tree (proof generation & verification)
│   │   ├── consensus.js        # Difficulty adjustment & chain validation
│   │   └── wallet.js           # secp256k1 key pairs, address derivation
│   │
│   └── services/
│       ├── aiPipeline.js       # 5-stage AI analysis (Gemini + fallback)
│       ├── assetService.js     # Asset lifecycle state machine
│       ├── tradingEngine.js    # Order book + matching + circuit breaker
│       ├── paymentService.js   # 4-gateway payment processing
│       ├── escrowService.js    # Double-entry escrow for asset funding
│       ├── kycService.js       # 4-tier KYC + AML transaction screening
│       ├── settlementService.js # Withdrawal processing + reconciliation
│       ├── priceService.js     # Price calculation engine
│       ├── feeService.js       # Fee collection (trading, listing, raise)
│       ├── complianceService.js # Pre-listing compliance checks
│       ├── documentProcessor.js # Document ingestion helpers
│       ├── eventBus.js         # In-memory pub/sub (14 event types)
│       └── wsServer.js         # WebSocket server with channels
│
├── frontend/
│   ├── index.html              # Main SPA (16KB)
│   ├── style.css               # Main styles (22KB)
│   ├── app.js                  # App logic, API calls (45KB)
│   ├── ws.js                   # WebSocket client
│   ├── admin.html              # Admin panel
│   ├── admin.css               # Admin styles
│   └── admin.js                # Admin logic (20KB)
│
├── data/
│   ├── averon.db               # SQLite database file
│   ├── chain.json              # Blockchain state (blocks + pending tx)
│   └── wallets.json            # ECDSA key store
```

```
│
├── scripts/
│   ├── seed.js                 # Seed demo data
│   ├── reset.js                # Reset database & chain
│   ├── benchmark.js            # Performance benchmarking
│   └── generate-history.js     # Generate realistic demo history
│
├── tests/
│   ├── blockchain.test.js      # Blockchain unit tests
│   ├── trading.test.js         # Trading engine tests
│   └── api.test.js             # API integration tests
│
├── uploads/                    # Asset document storage
└── logs/                       # Application logs
```

---

4. Blockchain Layer (In-Depth)

4.1 Architecture



4.2 Transaction Types

Type	Code	Description
`MINT`	Coin creation from fiat purchase	
`TRANSFER`	User-to-user or user-to-system	
`INVEST`	User → Escrow (buying asset tokens)	
`DIVEST`	Exit from investment	
`PAYOUT`	Escrow → Owner (asset fully funded)	
`REFUND`	Escrow → Investors (asset expired)	
`FEE`	Fee collection (trading, listing, capital raise)	
`ASSET_CREATE`	Asset tokenization recorded on chain	

Type	Code	Description
`ASSET_VERIFY`	AI verification result	
`ASSET_CLOSE`	Asset lifecycle completed	
`TRADE`	AC exchange between traders	
`REWARD`	Mining reward (0.1 AC per block)	

4.3 Block Mining

- **Algorithm**: SHA-256 Proof of Work
- **Difficulty**: Dynamic (adjusts every 10 blocks)
- **Target block time**: 30 seconds
- **Difficulty range**: 1–6 leading zeros
- **Max transactions per block**: 100
- **Max block size**: 1 MB
- **Mining reward**: 0.1 AC

4.4 Wallet System

- **Curve**: secp256k1 (same as Bitcoin/Ethereum)
- **Key format**: PEM (SPKI public, PKCS8 private)
- **Address derivation**: `SHA-256(publicKey) → RIPEMD-160 → 0x` prefix
- **Special wallets**:
  - `__SYSTEM__` — Mints coins, issues rewards
  - `__PLATFORM_FEE__` — Collects platform fees
  - Per-asset `ESCROW_<assetId>_<timestamp>` — Holds investor funds

4.5 Merkle Tree

Each block contains a Merkle root computed from all transaction hashes:

- Binary tree of SHA-256 hashes
- Odd leaves are duplicated
- Supports **proof generation** (`getProof()`) and **verification** (`verify()`)
- Enables efficient single-transaction verification without downloading the full block

4.6 Consensus Rules

- **Chain validation**: Index continuity, hash chain linkage, hash integrity, difficulty check, Merkle root verification, timestamp ordering
- **Difficulty adjustment**: Every 10 blocks — increases if blocks mine too fast (< expected/2), decreases if too slow (> expected×2)
- **Fork resolution**: Longest valid chain wins
- **Balance model**: UTXO-style scan across all blocks + pending outgoing deductions

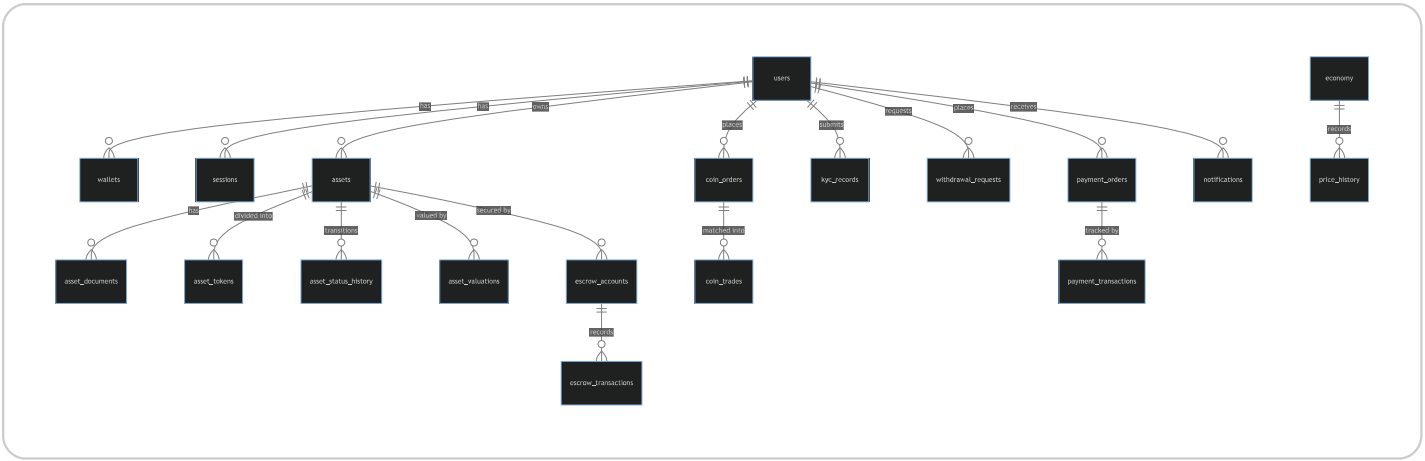
---

5. Database Layer (25+ Tables)

5.1 Technology

- **Engine**: sql.js (SQLite compiled to WebAssembly)
- **Persistence**: Auto-saves to `data/averon.db` every 3 seconds
- **Transactions**: Supports `BEGIN/COMMIT/ROLLBACK`
- **Zero native deps**: No C compilation needed

5.2 Schema Overview



5.3 Table Inventory

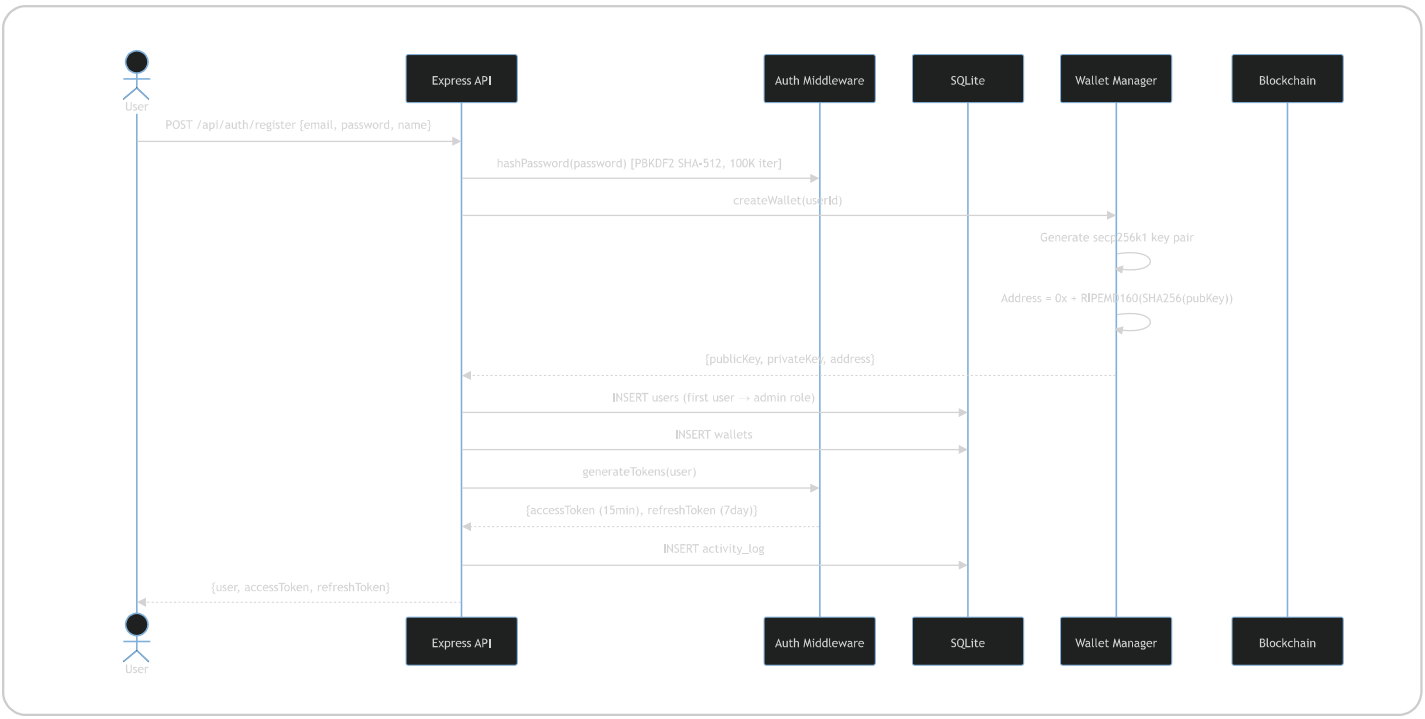
#	Table	Purpose	Key Columns
1	`users`	User accounts	id, email, password_hash, role, wallet_address, is_frozen, kyc_status
2	`sessions`	JWT refresh sessions	user_id, refresh_token, expires_at, is_revoked
3	`wallets`	ECDSA key pairs	user_id, public_key, private_key, address
4	`assets`	Tokenized assets	owner_id, title, category, status, ai_*, token_count, token_price, escrow_*, compliance_*
5	`asset_documents`	Uploaded docs	asset_id, filepath, mimetype, doc_hash
6	`asset_status_history`	State transitions	asset_id, old_status, new_status, changed_by, reason
7	`asset_valuations`	AI valuation records	asset_id, valuation, risk_score, confidence, source
8	`asset_tokens`	Fractional tokens	asset_id, token_index, price, owner_id, tx_hash

#	Table	Purpose	Key Columns
9	`escrow_accounts`	Escrow for each asset	asset_id, address, balance, total_received/released/refunded
10	`escrow_transactions`	Escrow movements	type (LOCK/RELEASE/REFUND), user_id, amount, tx_hash
11	`coin_orders`	Trading order book	user_id, side (buy/sell), type (market/limit), amount, price, filled, remaining, status
12	`coin_trades`	Executed trades	buy_order_id, sell_order_id, amount, price, buyer_fee, seller_fee, tx_hash
13	`economy`	Global metrics	price, total_supply, circulating_supply, total_raised_inr, market_cap, tvl
14	`price_history`	Price time series	price, volume, high, low, open, close, recorded_at
15	`fee_ledger`	All collected fees	user_id, fee_type, amount, reference_id
16	`notifications`	User notifications	user_id, type, title, message, is_read
17	`payment_gateways`	Gateway configs	name, provider, supported_currencies, min/max_amount, fees
18	`payment_orders`	Fiat→AC purchases	user_id, gateway, fiat_amount, coin_amount, exchange_rate, status, kyc_tier
19	`payment_transactions`	Payment audit trail	order_id, type, gateway, gateway_tx_id, status

#	Table	Purpose	Key Columns
20	`kyc_records`	KYC documents	user_id, doc_type, doc_number, doc_status, current_tier
21	`kyc_tier_history`	Tier changes	user_id, old_tier, new_tier, reason
22	`withdrawal_requests`	AC→Fiat withdrawals	user_id, coin_amount, fiat_amount, bank_account, status
23	`settlement_batches`	Batch settlements	gateway, total_amount, status
24	`reconciliation_log`	Payment reconciliation	order_id, internal_amount, gateway_amount, difference
25	`daily_limits`	KYC limit tracking	user_id, date, total_bought, tx_count
26	`audit_log`	Tamper-proof audit	action, details, prev_hash, entry_hash (hash chain!)
27	`system_config`	Dynamic config	key, value, updated_by
28	`activity_log`	User activity feed	user_id, action, details, tx_hash, amount

6. Service Layer — All Major Flows

6.1 User Registration & Authentication Flow

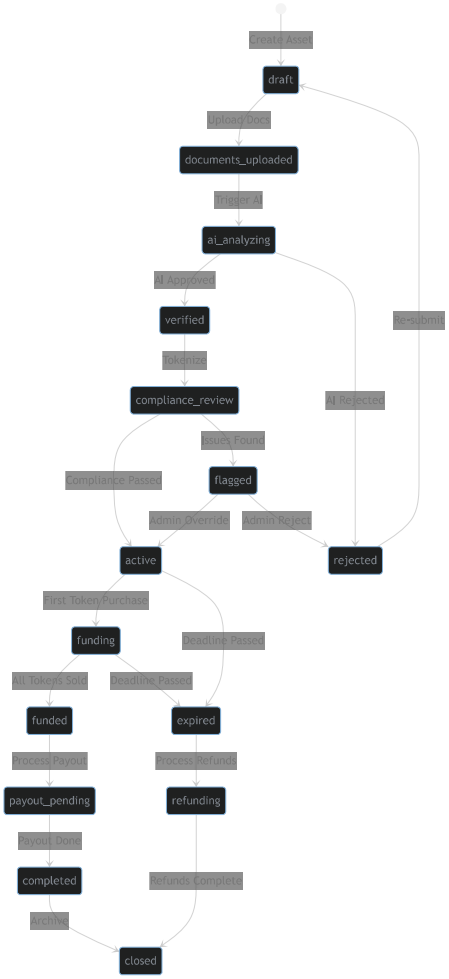


Key details:

- Passwords hashed with **PBKDF2-SHA512** (100K iterations, 64-byte key, 32-byte random salt)
- JWT tokens are **zero-dependency** — implemented from scratch with `crypto.createHmac('sha256')`
- Access tokens expire in **15 minutes**, refresh tokens in **7 days**
- First registered user automatically becomes **admin**
- Account lockout after **5 failed attempts** (15-minute lockout)
- Accounts can be **frozen** by admin

6.2 Asset Tokenization Lifecycle (State Machine)

This is the heart of Averon — a **14-state finite state machine** that governs every asset:



Full flow walkthrough:

Step 1: Asset Creation (`draft`)

- User provides: title, description, category (12 options), raise amount (₹100 – ₹1Cr), listing duration
- Validation: category must be from predefined list, raise amount within limits
- Asset inserted into DB with `draft` status

Step 2: Document Upload (`documents_uploaded`)

- User uploads 1–10 documents (JPG, PNG, WebP, PDF; max 10MB each)
- Files stored in `uploads/<assetId>/` with crypto-randomized filenames
- SHA-256 hash computed for duplicate detection
- Status transitions to `documents_uploaded`

Step 3: AI Analysis (`ai_analyzing` → `verified` / `rejected`)

The AI Pipeline runs **5 stages**:

Stage	Name	What It Does
1	**Document Ingestion**	Classify files, compute quality score (doc count, size, types)
2	**Duplicate Check**	SHA-256 hash comparison against all existing documents in DB
3	**AI Analysis**	Gemini 2.0 Flash multimodal (images + prompt) OR fallback engine
4	**Fraud Detection**	Duplicate docs (critical!), raise vs value ratio, low confidence, extreme risk
5	**Tokenization Recommendation**	Suggested token count and price based on raise amount and risk

Gemini prompt: Sends asset metadata + document images to `gemini-2.0-flash:generateContent` and expects JSON with `verified`, `estimated_value`, `risk_score`, `risk_level`, `analysis`, `concerns`, `confidence`.

Fallback engine: Uses category-specific profiles (e.g., Land: avg ₹5L, risk 10–35%) with randomized adjustments for document quality, raise-to-value ratio.

Verification criteria: `verified = true` AND `confidence ≥ 50` AND no critical fraud flags.

Step 4: Tokenization & Compliance (`verified` → `compliance_review` → `active`)

- Token count calculated (2–10,000 tokens, risk-adjusted)
- Token price = raise_amount / token_count (in both INR and AC)
- Escrow account created with unique address
- Compliance checks run:
  - Raise amount within limits
  - Minimum documents present
  - Description length check
- **Blockchain transaction**: `ASSET_CREATE` tx signed by owner, mined into block

- Asset transitions to `active` if compliance passes

Step 5: Token Investment (`active` → `funding` → `funded`)

- Investors spend AC to buy tokens
- Flow: User wallet → Escrow (INVEST transaction on blockchain)
- Tokens claimed atomically with `UPDATE WHERE owner_id IS NULL`
- Race condition protection: if fewer tokens claimed than expected, all are rolled back
- First purchase transitions asset to `funding`
- When all tokens sold → triggers `processFullyFunded()`

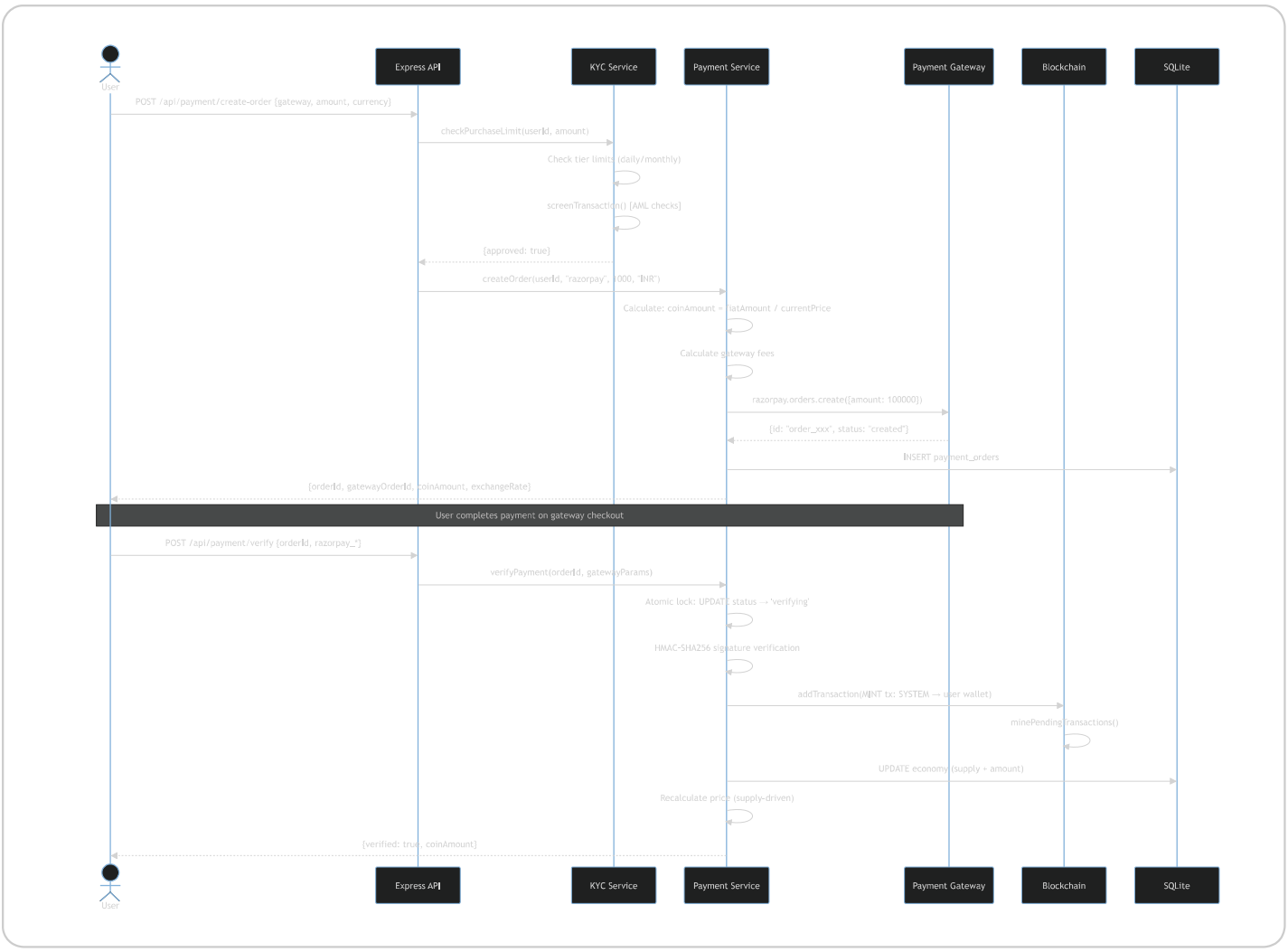
Step 6: Payout (`funded` → `payout_pending` → `completed`)

- Escrow released: platform takes 1% fee → `__PLATFORM_FEE__` wallet
- Remaining 99% paid to asset owner via blockchain PAYOUT tx
- AC price boosted by 2–5% (based on raise amount)
- Economy stats updated

Step 7: Expiry & Refund (`expired` → `refunding` → `closed`)

- Deadline checked every 60 seconds
- If expired and not fully funded: all investors refunded from escrow
- Each investor gets blockchain REFUND tx
- Token ownership cleared

6.3 Payment & Coin Minting Flow



Payment Gateways:

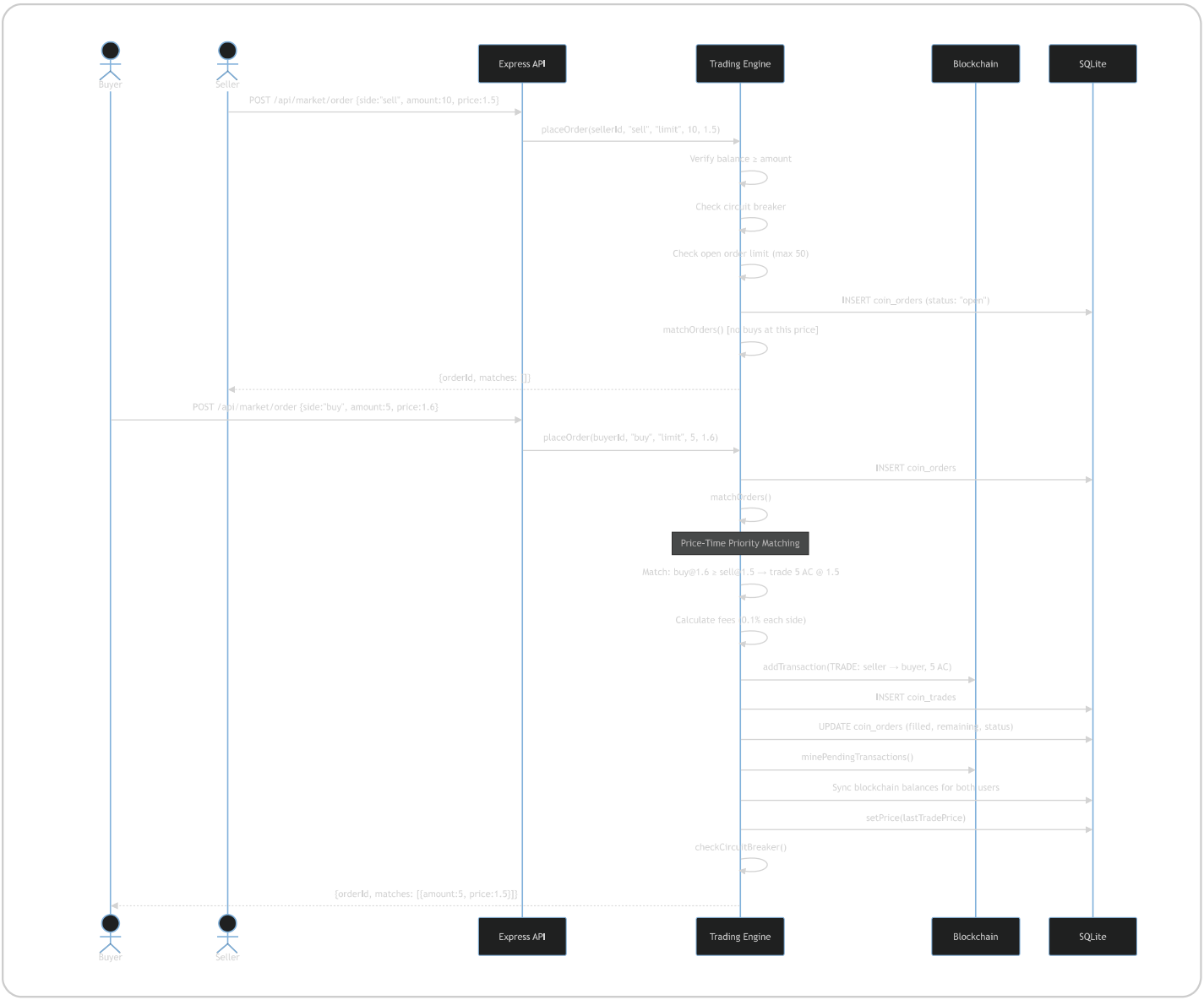
Gateway	Provider	Currencies	Min–Max	Verification
Razorpay	razorpay	INR	₹1 – ₹5Cr	HMAC-SHA256 signature
Stripe	stripe	INR,USD,EUR,GBP	₹1 – ₹10Cr	PaymentIntent status
UPI	upi	INR	₹1 – ₹2L	Admin confirmation
Wire	wire	INR,USD,EUR,GBP,SGD,AED	₹1L – ₹1000Cr	Admin confirmation

Double-mint prevention: Atomic `UPDATE ... WHERE status IN ('created','pending')` locks the order before verification — a second request gets `changes=0` and throws an error.

Price recalculation formula:

```
newPrice = INITIAL_PRICE × (1 + totalSupply/10000) × (1 + totalAssetsFunded × 0.04)
```

6.4 Trading Engine Flow



Order matching rules:

- **Price-Time Priority**: Best price first, then earliest order
- **Maker price wins**: Sell order price is the trade price (maker gets their price)
- **No self-trade**: `buy.user_id === sell.user_id` is skipped
- **Partial fills**: Orders can be partially filled across multiple matches
- **Market orders**: Use best available price or current economy price

Circuit breaker:

- Triggers when price moves >10% within 1 hour
- Halts all trading until the window resets
- Configurable via admin `system_config`

6.5 KYC & AML Flow

4-Tier KYC System:

Tier	Label	Daily Limit	Monthly Limit	Annual Limit	Requirements
0	Unverified	₹0	₹0	₹0	— (auto-granted in dev mode)
1	Basic KYC	₹1L	₹10L	₹50L	1 verified doc, 3 trades, ₹10K volume, 7 days
2	Full KYC	₹10L	₹1Cr	₹5Cr	10 trades, ₹1L volume, 30 days
3	Institutional	₹10Cr	₹100Cr	₹1000Cr	50 trades, ₹1Cr volume, 90 days

Supported documents: Aadhaar, PAN, Passport, Driving License, Voter ID, GST Certificate, Incorporation Certificate

AML Transaction Screening (runs on every purchase):

Flag	Trigger
`HIGH_VALUE`	Amount ≥ ₹10L
`ROUND_AMOUNT`	Round amount ≥ ₹1L (e.g., ₹100000, ₹200000)
`RAPID_SEQUENTIAL`	3+ transactions in 60 seconds
`NEW_ACCOUNT_HIGH_VALUE`	≥ ₹5L within 7 days of account creation
`HIGH_FREQUENCY`	Too many transactions in period
`DUPLICATE_PATTERN`	Repeated identical amounts
`STRUCTURING`	Splitting to avoid thresholds

Rule: Transaction is **blocked** if 3+ flags are triggered simultaneously.

6.6 Withdrawal & Settlement Flow

1. User requests withdrawal (min KYC Tier 1)
2. System checks blockchain balance
3. Withdrawal fee: 0.5 AC deducted
4. Net amount calculated at current exchange rate
5. Request enters `pending` status
6. **Admin processes** withdrawal:
   - Signs blockchain TRANSFER tx (user → system wallet)

- Signs FEE tx (user → platform fee wallet)
- Mines transactions
- Updates DB balances
- Decrements `circulating_supply` and `total_supply`

7. Admin marks as `completed` after bank transfer
8. **Reconciliation**: Periodically checks for discrepancies between internal records and gateway records

---

6.7 Admin Operations

The admin panel (`/admin.html`) provides:

Feature	API Endpoint	Description
Dashboard stats	`GET /api/admin/stats`	Economy, chain info, audit integrity, pending assets
Freeze account	`POST /api/admin/freeze/:userId`	Locks user from all operations
Unfreeze account	`POST /api/admin/unfreeze/:userId`	Restores user access
System config	`POST /api/admin/config`	Change fees, circuit breaker, etc.
KYC review	`GET /api/kyc/pending`	View pending KYC submissions
KYC approve/reject	`POST /api/kyc/verify/:recordId`	Approve or reject with reason
Pending withdrawals	`GET /api/withdraw/pending`	View withdrawal queue

Feature	API Endpoint	Description
Process withdrawal	`POST /api/withdraw/process/:id`	Execute on blockchain
Complete withdrawal	`POST /api/withdraw/complete/:id`	Mark completed
Fail withdrawal	`POST /api/withdraw/fail/:id`	Reject withdrawal
Refund payment	`POST /api/payment/refund`	Reverse a completed order
Audit chain integrity	(included in stats)	Verify hash chain of entire audit log

7. Middleware Stack

All requests pass through this pipeline:

```
Request → express.json (5MB limit) → sanitizeBody → generalLimiter → auditMiddleware →
```

7.1 Authentication

Middleware	Description
`authenticate`	Requires valid Bearer JWT; attaches `req.user`
`optionalAuth`	Attaches `req.user` if token present, continues if not
`requireRole(role)`	Checks `req.user.role` matches (e.g., `admin`)

Middleware	Description
`requireAdmin`	Shorthand for `requireRole('admin')`

7.2 Rate Limiting (Token Bucket)

Limiter	Window	Max Requests	Applied To
`generalLimiter`	60s	100	All routes
`authLimiter`	60s	10	Login, register
`financialLimiter`	1s	5	Buy, sell, trade, withdraw
`uploadLimiter`	60s	10	Document uploads

7.3 Tamper-Proof Audit Log

Every state-changing request (POST/PUT/PATCH/DELETE) is **hash-chained**:

```
entry_hash = SHA-256(prev_hash : user_id : action : resource_type : resource_id : times
```

The audit chain can be verified at any time via `verifyAuditChain()` — if any entry has been tampered with, the hash chain breaks and the corruption is detected and reported.

7.4 Input Validation

Validates and sanitizes all inputs for:

- `register` : email format, password strength (min 8 chars), name length
- `login` : email + password presence
- `createAsset` : title, category, raiseAmount bounds
- `buyTokens` : positive count

- `placeOrder` : side, amount, price validation

Sanitization strips `<script>` , `$` , `{` , `}` from all string body fields.

---

8. Real-Time Layer

8.1 Event Bus

In-memory pub/sub singleton with **14 event types**:

Event	Emitted When
`BLOCK_MINED`	New block mined
`TRADE_EXECUTED`	Order matched
`ORDER_PLACED`	New order submitted
`ORDER_CANCELLED`	Order cancelled
`PRICE_UPDATED`	Price changed (trade, mint, or fluctuation)
`ASSET_CREATED`	New asset listed
`ASSET_STATUS_CHANGED`	Asset state transition
`ASSET_FUNDED`	Asset fully funded
`TOKEN_PURCHASED`	Asset token bought
`COINS_MINTED`	AC minted from fiat
`USER_REGISTERED`	New user registered

Event	Emitted When
`NOTIFICATION`	Generic notification
`ESCROW_LOCKED` / `RELEASED`	Escrow state change
`CIRCUIT_BREAKER`	Trading halted/resumed

8.2 WebSocket Server

- **Channels**: `price` , `trades` , `blocks` , `assets` , `orders` , `users`
- **Client messages**: `subscribe` , `unsubscribe` , `auth` (JWT verification), `ping`
- **Server broadcasts**: Event bus events → channel-specific messages
- **Auth**: Optional JWT-based authentication for user-specific data

---

9. Price Engine

The AC coin price is driven by **three mechanisms**:

1. **Supply-driven recalculation** (on each purchase):

```
price = 1.0 × (1 + totalSupply/10000) × (1 + assetsFunded × 0.04)
```

2. **Trade-driven** (on each matched trade): price = last trade price

3. **Natural fluctuation** (every 15 seconds):

```
swing = random(-0.25%, +0.25%)
newPrice = max(0.001, currentPrice × (1 + swing))
```

4. **Asset funding boost** (when asset fully funded):

```
boost = min(5%, 2% + raiseAmount/1M × 3%)
```

Price history is recorded in the `price_history` table for charting.

---

10. Fee Structure

Fee Type	Rate	Collected By
Trading fee	0.1% per trade (both sides)	`__PLATFORM_FEE__` wallet
Asset listing fee	1.0 AC	(configured, not currently enforced)
Capital raise fee	1.0% of raised amount	Deducted from escrow payout
Withdrawal fee	0.5 AC	Sent to `__PLATFORM_FEE__` wallet
Gateway fees	Varies by gateway	Deducted from fiat amount

All fees are recorded in the `fee_ledger` table and aggregated in `economy.total_fees_collected`.

---

11. Security Model

Aspect	Implementation
Password storage	PBKDF2-SHA512, 100K iterations, 32-byte random salt
JWT tokens	HMAC-SHA256, zero-dependency, short-lived access (15min)
Blockchain signing	ECDSA secp256k1 (Bitcoin-grade)
Audit trail	SHA-256 hash-chained, tamper-detectable
Rate limiting	4-tier token bucket (100/10/5/10 req per window)
Input sanitization	HTML entity stripping, length limits
Account lockout	5 failed logins → 15-minute lock
Account freezing	Admin can freeze any account
Double-mint prevention	Atomic DB status lock before minting
Race condition protection	`UPDATE WHERE owner_id IS NULL` for token claims
Webhook verification	HMAC signature validation (Razorpay/Stripe)
Circuit breaker	10% price move in 1 hour halts trading
KYC/AML	4-tier limits + 8 AML flag types, blocks at 3+ flags
Security headers	Helmet.js (CSP, HSTS, XSS protection)
CORS	Configurable allowed origins

12. Deployment Architecture

Development

```
npm run dev          # Node --watch mode (auto-restart on changes)
```

Production (PM2)

```
npm run production   # PM2 cluster mode with ecosystem.config.js
                     # Auto-forks N workers (= CPU count)
                     # Auto-restarts on crash
```

Docker

```
docker compose up -d  # Single container, health checks every 15s
                      # 1GB memory limit, 256MB reserved
                      # Persistent volumes for data/ and uploads/
```

Cluster mode (production): The primary process forks N workers using `cluster` module. Each worker runs the full Express app independently. Workers auto-restart on crash. Graceful shutdown propagates `SIGTERM` to all workers.

---

13. API Route Map (40+ endpoints)

Method	Path	Auth	Description
`GET`	`/api/config`	—	Platform config & stats

Method	Path	Auth	Description
GET	/api/dashboard	—	Dashboard stats + activity
POST	/api/auth/register	—	Register + wallet creation
POST	/api/auth/login	—	Login + JWT tokens
POST	/api/auth/refresh	—	Refresh access token
GET	/api/account	✅	Current user profile
GET	/api/notifications	✅	User notifications
POST	/api/notifications/read	✅	Mark all read
GET	/api/payment/gateways	✅	Available payment gateways
POST	/api/payment/create-order	✅ 💰	Create fiat→AC order
POST	/api/payment/verify	✅	Verify & mint coins
POST	/api/payment/refund	✅ 👑	Refund a completed order
GET	/api/payment/orders	✅	User's payment history
POST	/api/webhook/:gateway	—	Gateway webhook handler
POST	/api/kyc/submit	✅ 🕯️	Submit KYC document
GET	/api/kyc/status	✅	KYC tier & records
GET	/api/kyc/pending	✅ 👑	Admin: pending KYC
POST	/api/kyc/verify/:id	✅ 👑	Admin: approve/reject KYC
POST	/api/withdraw/request	✅ 💰	Request AC→fiat withdrawal

Method	Path	Auth	Description
GET	/api/withdraw/history	✅	Withdrawal history
GET	/api/withdraw/pending	✅ 👑	Admin: pending withdrawals
POST	/api/withdraw/process/:id	✅ 👑	Admin: process withdrawal
POST	/api/withdraw/complete/:id	✅ 👑	Admin: mark completed
POST	/api/withdraw/fail/:id	✅ 👑	Admin: reject withdrawal
GET	/api/assets	🔓	List all assets
GET	/api/assets/:id	🔓	Asset detail
POST	/api/assets/create	✅	Create new asset
POST	/api/assets/:id/documents	✅ 📎	Upload documents
POST	/api/assets/:id/analyze	✅	Trigger AI analysis
POST	/api/assets/:id/confirm	✅	Tokenize asset
POST	/api/assets/:id/tokens/buy	✅ 💰	Buy asset tokens
GET	/api/market/orderbook	—	Order book + trades
POST	/api/market/order	✅ 💰	Place buy/sell order
DELETE	/api/market/order/:id	✅	Cancel order
GET	/api/portfolio	✅	Full user portfolio
GET	/api/blockchain/info	—	Chain statistics
GET	/api/blockchain/blocks	—	Recent blocks

Method	Path	Auth	Description
GET	/api/blockchain/block/:idx	—	Block by index
GET	/api/blockchain/tx/:hash	—	Transaction by hash
GET	/api/blockchain/validate	—	Chain validity
GET	/api/blockchain/address/:addr	—	Address balance + history
GET	/api/admin/stats	✅ 👑	Full admin dashboard
POST	/api/admin/freeze/:userId	✅ 👑	Freeze account
POST	/api/admin/unfreeze/:userId	✅ 👑	Unfreeze account
POST	/api/admin/config	✅ 👑	Update system config
GET	/api/economy	—	Economy metrics
GET	/api/economy/price-history	—	Price time series
GET	/health	—	Health check

> *Legend: ✅ = Auth required, 💰 = Financial rate limit, 👑 = Admin only, 📎 = File upload, 🔐 = Optional auth*

---

14. Initialization Sequence

```
1. Load .env (dotenv)
2. Check cluster mode (production + primary → fork workers and exit)
```

```
3. Set up global error boundaries (uncaughtException, unhandledRejection)
4. Initialize Express app with middleware stack
5. Initialize SQLite WASM database
   └── Create 25+ tables (IF NOT EXISTS)
   └── Seed economy defaults and system_config
   └── Start auto-persist timer (every 3s)
6. Initialize blockchain
   └── Load chain.json or create genesis block
7. Initialize wallet manager
   └── Load wallets.json, ensure SYSTEM and PLATFORM_FEE wallets
8. Initialize all services (escrow, asset, trading, fee, price, compliance, KYC, paymen
9. Start background timers
   └── Price fluctuation (every 15s)
   └── Asset deadline check (every 60s)
   └── Session cleanup (every 1h)
10. Create HTTP server + WebSocket server
11. Listen on PORT (default 4200)
12. Register graceful shutdown handlers (SIGTERM, SIGINT)
    └── Broadcast shutdown via WebSocket
    └── Clear all timers
    └── Persist database
    └── Close HTTP server
    └── Force exit after 10s timeout
```

> *This document covers every module, every flow, every table, and every integration point in Averon v4.0.0.*